Poisson Model with an Offset

General Principles

When we want to model count data, where the counts are observed over different periods or areas of exposure, we use a *Poisson model with an offset*. This is a type of generalized linear model used for modeling count data and contingency tables.

An *offset* is a predictor variable with a coefficient that is fixed at 1. It is used to account for the "exposure" variable, which represents the opportunity for an event to occur. For instance, if we are counting the number of sick individuals in different cities, the population of each city would be the exposure variable. A city with a larger population is expected to have more sick individuals. The offset accounts for this by essentially modeling the rate of events per unit of exposure.

Considerations

> **Note**
>
> - The dependent variable in a Poisson regression must be a non-negative count.
> - The exposure variable used as an offset cannot contain zeros.
> - A key assumption of the Poisson distribution is that the mean and variance of the count variable are equal. If the variance is greater than the mean, a condition known as overdispersion, a Negative Binomial regression might be more appropriate.
> - The logarithm of the exposure variable is typically used as the offset. This is because Poisson regression models the logarithm of the expected count. By including the log of the exposure as an offset, we are effectively modeling the rate.

Example

Below is an example of code that demonstrates a Bayesian Poisson regression with an offset. The data consists of the number of elephant aggressions (`agressions`), the age of the elephants (`age`), and the number of years they have been observed (`years_obs`). The goal is to model the rate of aggressions per year, accounting for the age of the elephants.

Python

```python
from BI import bi
import jax.numpy as jnp
# Setup device-------------------------------------------------
m = bi(platform='cpu')

# Simulated data -----------------------------------------------
population = m.dist.normal(0,1, shape = (100,), sample = True)
cid = m.dist.binomial(1, 0.5, shape = (100,), sample = True)
hours = m.dist.uniform(1, 10, shape=(100,), sample=True)

a = m.dist.normal(3, 0.5, shape= (2,), name='a', sample = True)
b = m.dist.normal(0, 0.2, shape=(2,), name='b', sample = True)
l = hours * jnp.exp(a[cid] + b[cid]*population)
total_tools = m.dist.poisson(l,  sample = True)

# Model data ---------------------------------------------------
def model_offset(cid, population, hours, total_tools):
    a = m.dist.normal(3, 0.5, shape= (2,), name='a')
    b = m.dist.normal(0, 0.2, shape=(2,), name='b')
    l = hours * jnp.exp(a[cid] + b[cid]*population)
    m.dist.poisson(l, obs=total_tools)

m.data_on_model = dict(cid=cid, population=population, hours=hours, total_tools=total_tools)

# Run sampler --------------------------------------------------
m.fit(model_offset, progress_bar=False)

# Diagnostic ---------------------------------------------------
m.summary()
```

R

```r
library(BayesianInference)
m=importBI(platform='cpu')
jnp = reticulate::import('jax.numpy')

# Simulated data ------------------------------------------------
population = bi.dist.normal(0,1,shape = c(100), sample = TRUE)
cid = bi.dist.binomial(probs = c(0.5), shape = c(100),sample = TRUE)
hours = bi.dist.uniform(1, 10, shape=c(100), sample=TRUE)

a = bi.dist.normal(3, 0.5, shape= (2), name='a', sample = TRUE)
b = bi.dist.normal(0, 0.2, shape=(2), name='b', sample = TRUE)
l = hours * jnp$exp(a[cid] + b[cid]*population)
total_tools = bi.dist.poisson(l,  sample = TRUE)

# Define model ------------------------------------------------
model <- function(cid, population, hours, total_tools_offset){
  # Parameter prior distributions
  alpha = bi.dist.normal(3, 0.5, name='alpha', shape = c(2))
  beta = bi.dist.normal(0, 0.2, name='beta', shape = c(2))
  l = hours * jnp$exp(alpha[cid] + beta[cid]*population)
  # Likelihood
  bi.dist.poisson(l, obs=total_tools)
}

m$data_on_model = list()
m$data_on_model$total_tools = total_tools
m$data_on_model$population = population
m$data_on_model$cid = cid
m$data_on_model$hours = hours

# Run mcmc ------------------------------------------------
m$fit(model) # Optimize model parameters through MCMC sampling

# Summary ------------------------------------------------
m$summary() # Get posterior distributions
```

Julia

```julia
using BayesianInference

# Setup device-------------------------------------------------
m = importBI(platform="cpu")

# Simulated data ----------------------------------------------
population = m.dist.normal(0, 1, shape = (100,), sample = true)
cid = m.dist.binomial(1, 0.5, shape = (100,), sample = true)
hours = m.dist.uniform(1, 10, shape=(100,), sample=true)

a = m.dist.normal(3, 0.5, shape= (2,), name="a", sample = true)
b = m.dist.normal(0, 0.2, shape=(2,), name="b", sample = true)
l = hours * jnp.exp(a[cid] + b[cid]*population)
total_tools = m.dist.poisson(l,  sample = true)

# Define model ------------------------------------------------
@BI function model(cid, population, hours, total_tools)
    a = m.dist.normal(3, 0.5, shape= (2,), name="a")
    b = m.dist.normal(0, 0.2, shape=(2,), name="b")
    l = hours * jnp.exp(a[cid] + b[cid]*population)
    m.dist.poisson(l, obs=total_tools)
end

# Pass data to model
m.data_on_model = Dict("cid" => cid, "population" => population, "hours" => hours, "total_too

# Run mcmc ----------------------------------------------------
m.fit(model)  # Optimize model parameters through MCMC sampling

# Summary -----------------------------------------------------
m.summary() # Get posterior distributions
```

Mathematical Details

Frequentist formulation

We model the relationship between the independent variables (X) and the expected count ( ) using the following equation:

$$\log(\lambda_i) = \alpha + \beta X_i + \log(\text{exposure}_i)$$

Where:

- λ_i is the expected count for observation i.
- α is the intercept term.
- β is the regression coefficient for the independent variable.
- X_i is the value of the independent variable for observation i.
- $\log(\text{exposure}_i)$ is the offset, which is the natural logarithm of the exposure for observation i.

The number of observed counts Y_i is assumed to follow a Poisson distribution with mean λ_i:

$$Y_i \sim \text{Poisson}(\lambda_i)$$

Bayesian formulation

In the Bayesian framework, we assign prior distributions to the model parameters. The model can be expressed as:

$$Y_i \sim \text{Poisson}(\lambda_i)$$

$$\log(\lambda_i) = \alpha + \beta X_i + \log(\text{exposure}_i)$$

$$\alpha \sim \text{Normal}(0, 1)$$

$$\beta \sim \text{Normal}(0, 1)$$

Where:

- Y_i is the observed count for observation i.
- λ_i is the expected count.
- α is the intercept with a unit-normal prior.
- β is the slope coefficient with a unit-normal prior.
- X_i is the independent variable.
- $\log(\text{exposure}_i)$ is the offset.

Notes

> **Note**
>
> - The use of an offset is crucial when the goal is to compare rates of events rather than absolute counts.
> - It is a common practice to use the natural logarithm of the exposure variable as the offset.

Reference(s)